

PROJECT REPORT
On
CHATHIVE: Real-Time Chat-Application

Submitted in partial fulfilment of the requirement for the

Course COOP Module of

COMPUTER SCIENCE AND ENGINEERING
B.E. Batch-2022



Under the Guidance of:
Dr. Shyam Goyal

Submitted By:
Abhishek(2210991164)

Kashish(2210991767)

Kashish(2210990495)

Arnav(2210991349)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CHITKARA UNIVERSITY PUNJAB

CERTIFICATE

This is to be certified that the project entitled “**CHATHIVE: Real-Time Chat-Application**” has been submitted for the Bachelor of Computer Science Engineering at Chitkara University, Punjab during the academic semester January 2025- May-2025 is a bonafide piece of project work carried out by **Abhishek(2210991164), Kashish(2210991767), Kashish(2210990495), Arnav(2210991349)** towards the partial fulfilment for the award of the course COOP Module.

CANDIDATE'S DECLARATION

We, **Abhishek(2210991164), Kashish(2210991767), Kashish(2210990495), Arnav(2210991349)** of B.E.-2022 of the Chitkara University, Punjab hereby declare that the COOP Module Report entitled **“CHATHIVE: Real-Time Chat-Application”** is an original work and data provided in the study is authentic to the best of our knowledge. This report has not been submitted to any other Institute for the award of any other course.

Sign. of Student 1

Kashish Barthwal

2210990495

Sign. of Student 2

Arnav Jain

2210991349

Sign. of Student 3

Abhishek

2210991164

Sign. of Student 4

Kashish

2210991767

ACKNOWLEDGEMENT

It is our pleasure to be indebted to various people, who directly or indirectly contributed in the development of this work and who influenced my thinking, behaviour and acts during the course of study.

We express our sincere gratitude to all for providing me an opportunity to undergo Integrated Project as the part of the curriculum.

We are thankful to Dr. Shyam Goyal for his support, cooperation, and motivation provided to us during the training for constant inspiration, presence and blessings.

Lastly, we would like to thank the almighty and our parents for their moral support and friends with whom we shared our day-to day experience and received lots of suggestions that improve our quality of work.

Sign. of Student 1

Kashish Barthwal

2210990495

Sign. of Student 2

Arnav Jain

2210991349

Sign. of Student 3

Abhishek

2210991164

**Sign. of Student
4**

Kashish

2210991767

Index

1. Abstract/Keywords	6
2. Introduction to the project	7-9
2.1 Background	
2.2 Problem Statement	
3. Software and Hardware Requirement Specification	10-11
3.1 Software Requirements	
3.2 Hardware Requirements	
3.3 Methods	
3.4 Programming / Working Environment	
3.5 Requirements to Run the Application	
4. Dataset Analysis, Design, and Implementation	12
5. Program's Structure Analyzing and GUI Constructing	13-15
6. Code Implementation and Dataset Integration	16
7. Conclusion	17
8. Future Scope	18
9. Bibliography/References	19

1. Abstract

ChatHive is a full-stack real-time chat application designed to enable seamless and instant communication between users. It uses **Socket.IO** to establish WebSocket connections, allowing real-time message exchange without needing page reloads. Users can send and receive both text messages and images, making conversations interactive and modern. For image sharing, the app integrates **Cloudinary**, which efficiently manages and stores uploaded media files.

User authentication in ChatHive is handled securely using **httpOnly cookies**, ensuring protection against common web vulnerabilities like XSS. Additionally, a **forgot password** feature is implemented using **Nodemailer**, allowing users to recover access to their accounts via email in case of credential loss. The backend, developed in **Node.js** with **Express**, is responsible for managing API routes, socket connections, and authentication logic, while **MongoDB** stores user data and chat history.

The frontend is built using **React.js** and styled with **Tailwind CSS**, enhanced further by **DaisyUI** for customizable UI themes. Users can toggle between light and dark modes or other themes based on preference. The interface is responsive and designed to provide a smooth experience across devices. ChatHive also displays online users in real time, helping participants know who is active, thus making the platform more engaging and collaborative.

ChatHive is scalable, secure, and suitable for various real-world use cases including personal communication, internal team collaboration, and educational discussions. This report outlines the system's design, implementation strategies, chosen technologies, and the challenges overcome during development. With future enhancements like video calling or emotion-based feedback systems, ChatHive lays the foundation for more immersive digital communication experiences.

2. Introduction to the Project

Chat Application

Subtitle: Developing a Secure and Efficient Web-Based Chat System

The project focuses on building a real-time chat application that allows users to communicate instantly through a web-based platform. In today's fast-paced world, instant messaging has become a fundamental tool for communication, whether for personal conversations, team collaboration, or customer support. This project aims to create a chat application that not only facilitates real-time communication but also ensures security, scalability, and a seamless user experience.

The core features of the application include:

- **Instant Message Delivery:** Using Socket.io, the application enables real-time bidirectional communication, ensuring that messages are delivered instantly without delays. Unlike traditional HTTP requests, which rely on a request-response model, WebSockets allow the server to push updates to the client as soon as they are available. This ensures a smooth and responsive chatting experience.
- **Secure Authentication:** The application implements JWT (JSON Web Tokens) and cookies to authenticate users and protect their data. JWT provides a secure way to transmit information between the client and server, ensuring that only authorized users can access the chat system. Additionally, the application employs encryption techniques to protect sensitive data during transmission.
- **Responsive User Interface:** The frontend is built using React.js, a popular JavaScript library for building user interfaces. React.js ensures that the application is highly responsive and provides a smooth user experience. The application is styled using Tailwind CSS, a utility-first CSS framework that allows for rapid development of modern and responsive designs.
- **Persistent Message Storage:** Messages are stored in MongoDB, a NoSQL database, allowing users to retrieve their chat history at any time. MongoDB's document-based structure makes it an ideal choice for storing chat messages, as it allows for flexible and efficient data storage.

2.1 Background

In today's digitally connected world, communication platforms have evolved significantly, with instant messaging emerging as one of the most essential tools for both personal and professional interaction. Whether it's used in remote work environments, online education systems, or day-to-day conversations, users expect real-time, secure, and seamless communication. The demand for scalable and responsive chat applications has grown rapidly, especially with the global shift toward remote collaboration and online services.

Despite the availability of various messaging tools, many existing platforms either lack customization, are overloaded with unnecessary features, or fall short in delivering real-time performance and strong data protection. Traditional HTTP-based communication often introduces latency, making it unsuitable for applications requiring instant updates. Additionally, chat applications must address key challenges such as efficient data storage, user authentication, and UI responsiveness across devices.

To meet these growing expectations, this project proposes the development of a full-stack web-based chat application built with React.js on the frontend and Node.js/Express.js on the backend. It utilizes Socket.io to enable real-time, bidirectional messaging, while MongoDB is used to manage and persist chat history effectively. Security is enforced through JWT-based authentication and cookie handling, ensuring that users' data and sessions are protected.

This background sets the stage for a solution that not only meets but exceeds current user expectations by delivering a robust, scalable, and responsive communication platform—built with modern web technologies and best practices in full-stack development.

2.2 Problem Statement

One of the most significant challenges is ensuring real-time communication without delays. Users expect instant message delivery, which requires a robust mechanism for bidirectional communication. Traditional HTTP requests are unsuitable due to their request-response model, introducing latency. To overcome this, the project leverages WebSockets, specifically Socket.io, to enable full-duplex communication and seamless message delivery.

Another critical challenge is data storage. The application must efficiently store and retrieve chat history. MongoDB, a NoSQL database, is chosen for its flexibility, scalability, and document-based structure, making it ideal for handling large volumes of chat data.

Security is a major concern, requiring proper authentication mechanisms. The project uses JSON Web Tokens (JWT) and cookies to authenticate users and protect data. Encryption techniques are also employed to secure sensitive data during transmission.

Scalability is essential to handle multiple concurrent users without performance issues. The backend, built with Node.js and Express, ensures efficient handling of simultaneous connections. Load balancing and database optimization are also considered.

Finally, cross-browser compatibility is ensured by testing the application across various browsers and devices, providing a consistent user experience.

By addressing these challenges, the project delivers a reliable chat application using modern technologies like React.js, Node.js, Express, Socket.io, and MongoDB, showcasing how to build a scalable and secure real-time communication system.

3. Software and Hardware Requirement Specification

3.1 Software Requirements:

- Operating System: Windows, macOS, or Linux
- **Frontend:**
 - HTML, CSS, JavaScript
 - React.js (for component-based structure)
 - Tailwind CSS with **DaisyUI** (for UI styling and theme integration)
- **Backend:**
 - Node.js
 - Express.js (for building RESTful APIs)
- **Real-Time Communication:**
 - Socket.IO (for WebSocket-based real-time chat)
- **Database:**
 - MongoDB (for storing user data and messages)

3.2 Hardware Requirements:

- Processor: 2.0 GHz Dual Core or faster
- RAM: Minimum 4 GB (8 GB or more recommended)
- Storage: At least 1 GB free (for dataset and model checkpoints)
- Internet: Required for real-time communication, Cloudinary uploads, and Nodemailer functionality

3.3 Methods

3.3.1 Application Design Approach:

- **Modular Architecture:** Backend APIs are separated from real-time WebSocket events and frontend logic for maintainability.
- **Frontend UI/UX:** Built with React.js and styled using **DaisyUI**, which allows dynamic theme switching for better user experience.
- **Real-Time Messaging:** Implemented using **Socket.IO**, ensuring low-latency message delivery and room-based communication.
- **Media Handling:** Cloudinary integration allows users to send image files seamlessly without storage concerns.
- **Authentication Strategy:** Secure login system using cookies. Forgot password feature is implemented using **Nodemailer**, which sends reset links to registered emails.

3.3.2 Development & Testing Strategy:

- **Component Testing:** UI components are tested using browser dev tools and React testing libraries.
- **API Testing:** Postman is used to test backend APIs during development.
- **Error Handling:** Includes basic try-catch blocks in backend and frontend-level error notifications.
- **Security Measures:** Input validation, cookie-based session management, and secure image handling.

3.4 Programming / Working Environment

- **IDE:** Visual Studio Code (recommended for frontend and backend development)
- **Command Line Tools:** Terminal/Bash for package management and server control
- **Package Managers:**
 - npm or yarn for installing frontend/backend dependencies
- **Hosting Services:**
 - Vercel/Netlify (Frontend)
 - Railway/Render (Backend & Database)

3.5 Requirements to Run the Application

- **Install Node.js and npm** (<https://nodejs.org/>)
- **Clone the Repository** using Git:
 - o `git clone https://github.com/your-username/chathive.git`
 - o `cd chathive`
- **Install Backend Dependencies:**
 - o `cd backend`
 - o `npm install`
- **Install Frontend Dependencies:**
 - o `cd ../frontend`
 - o `npm install`
- **Configure Environment Variables** for Cloudinary, MongoDB, and Nodemailer in .env files
- **Run Backend Server:**
 - o `cd backend`
 - o `npm start`
- **Run Frontend Dev Server:**
 - o `cd ../frontend`
 - o `npm run dev`
- **Access Application at:** <http://localhost:3000>
- **Ensure MongoDB and Cloudinary credentials are correctly set** for storing chat messages and media

4. Dataset Analysis, Design, and Implementation

4.1 Dataset Analysis

ChatHive, being a real-time chat application, does not rely on a static dataset like traditional machine learning models. Instead, it handles **dynamic user-generated data** including messages, images, and session information. The application's backend is designed to manage structured data such as user credentials, chat history, online status, and media links, ensuring seamless real-time communication and data integrity.

Data sources in ChatHive include:

- User-generated messages and media
- Authentication and session tokens
- Real-time socket activity logs
- **Cloudinary URLs for uploaded images**

4.2 Dataset Design

The data architecture is built using **MongoDB**, with collections logically designed for efficiency and scalability. The primary collections include:

- Users: stores user details such as username, email, password(hash) and online status.
- Messages: **Stores individual messages with metadata including sender, receiver, timestamp, message type (text/image), and Cloudinary URLs for images.**

4.3 Dataset Implementation

The real-time implementation of data within ChatHive involves the following components:

- **Socket.IO** enables bidirectional communication between clients and server, allowing instantaneous message delivery and updates.
- **Cloudinary Integration** for storing and retrieving media files like images. Upon upload, the server receives a Cloudinary URL, which is stored in the messages collection.
- **Express.js & MongoDB** handle RESTful routes for user login, registration, message fetching, and status updates.
- **Nodemailer** supports password reset functionality by sending secure reset links via email.

5. Program's Structure Analysing and GUI Constructing

5.1 Program Structure

ChatHive follows a well-structured and modular architecture that separates concerns between backend logic, real-time communication, media handling, and frontend user experience. This separation ensures scalability, maintainability, and efficient real-time interactions across the platform. The application structure consists of the following layers:

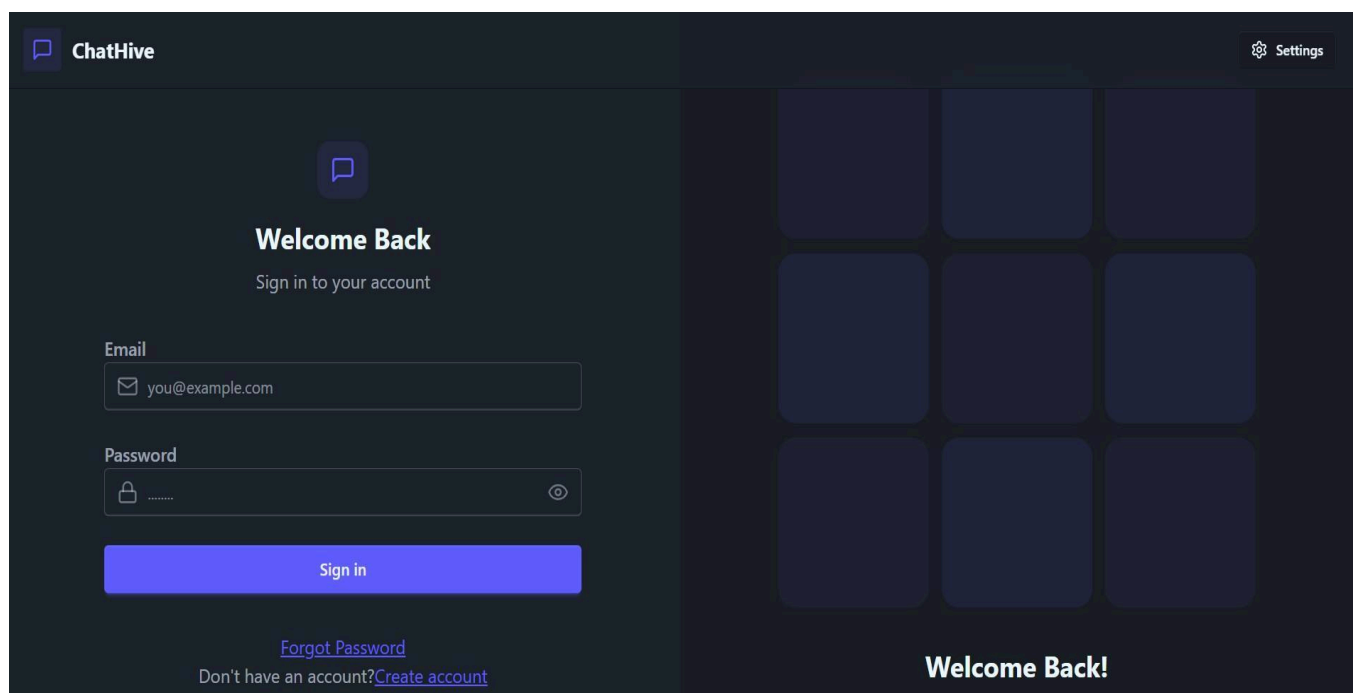
- **Backend Layer:** Built with **Node.js** and **Express.js**, this layer handles RESTful API endpoints for user authentication (registration, login, logout), message management, and image uploads. It manages persistent data using **MongoDB**, and authentication is implemented via **httpOnly cookies** for enhanced security. **Nodemailer** is used to handle password recovery through email-based reset links.
- **Socket Layer:** Real-time chat is powered by **Socket.IO**, which establishes WebSocket connections between users. This layer manages live message broadcasting, online user tracking, and room-specific message delivery. Users can see live updates when others join, leave, or send messages in a chat room.
- **Media & Cloud Integration Layer:** Image messages are supported through **Cloudinary**, a cloud-based image hosting service. When a user uploads a picture, the server uploads it to Cloudinary and returns a URL, which is saved and rendered in the chat. This offloads storage concerns and ensures fast content delivery.
- **Frontend Layer:** Developed using **DaisyUI** on top of **Tailwind CSS**, the frontend offers a responsive and aesthetic user interface. Multiple themes are integrated, allowing users to switch between different visual styles dynamically. Chat messages are displayed in real time, and the interface adapts based on screen size and user preferences.
- **Authentication and Session Management:** A robust login and registration system is implemented with proper error handling. Sessions are maintained using cookies. Forgot password functionality is implemented using a secure link system managed via **Nodemailer**.

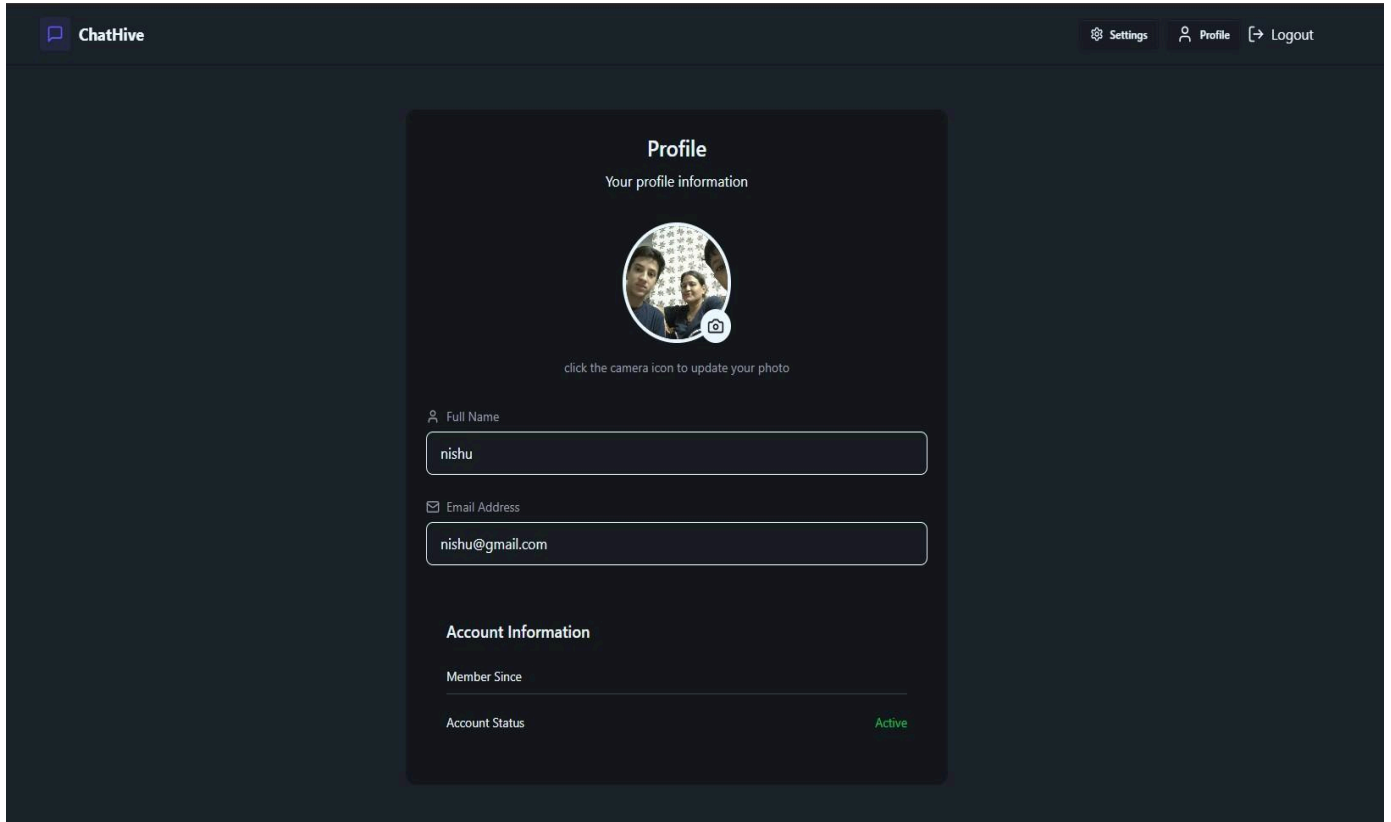
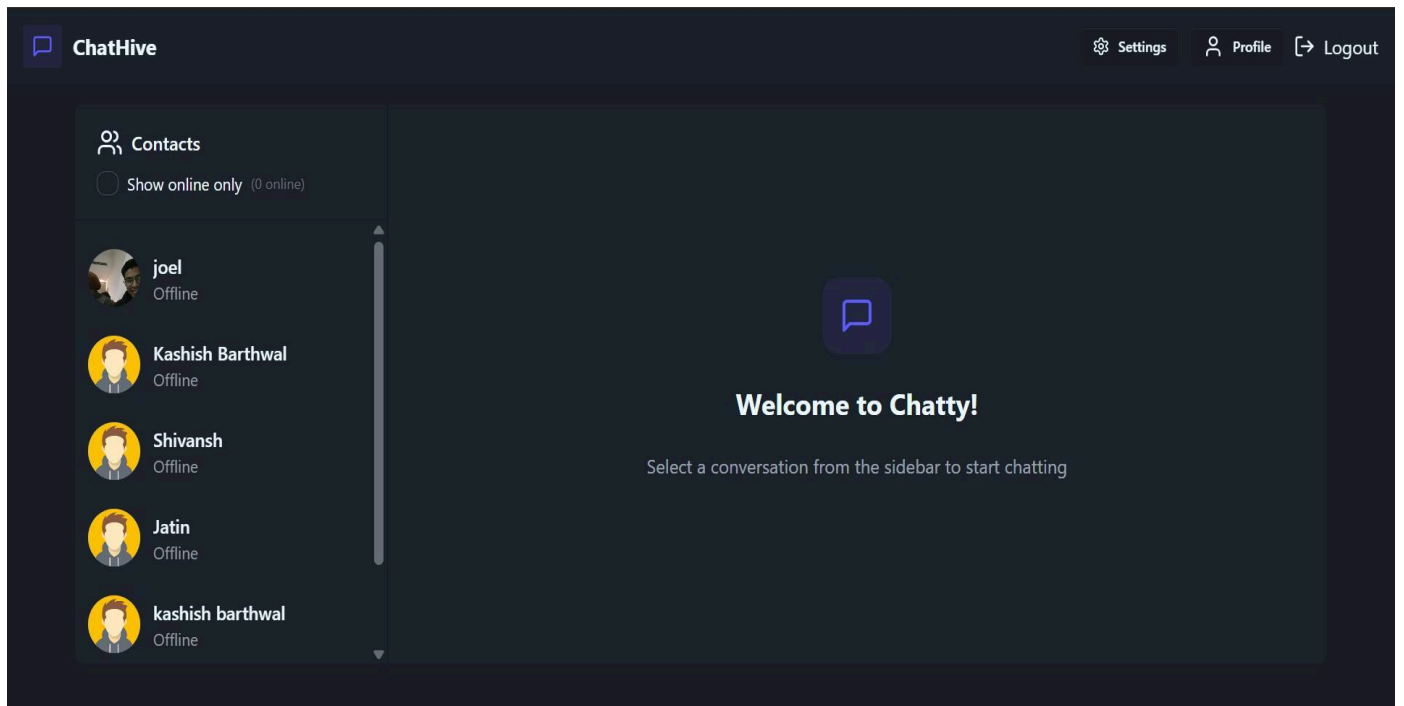
5.2 GUI Construction

The GUI of ChatHive is designed to deliver a smooth and interactive user experience across all devices. Using **DaisyUI**, a component-rich Tailwind plugin, the frontend features:

- A modern **login/register interface** with error validation and password visibility toggle.
- A **main chat dashboard** showing the list of online users, recent messages, and chat history.
- **Dynamic message area** that updates in real time without requiring a refresh.
- Support for both **text and image messages**, with image previews and responsive scaling.
- **Theme switching** options allowing users to personalize their experience using DaisyUI's built-in themes

This modular design and clean user interface make ChatHive a reliable, extensible, and engaging chat platform, ready for further enhancements like group chats, emoji support, or video calling integration.





6. Code Implementation and Dataset Integration:

6.1 Code Implementation

- **Real-Time Messaging with Socket.IO:** Chat Hive leverages **Socket.IO** to enable instant, real-time communication between users. Messages sent from one user are emitted to all connected clients, creating a smooth and responsive chat experience with minimal delay.
- **Authentication with Cookies:** The authentication system is handled using **cookie-based sessions**. Once users log in, a secure token is stored in cookies, which are then used by the server to verify and maintain user sessions. This ensures both usability and protection against unauthorized access.
- **Image Upload via Cloudinary:** Chat Hive allows users to send images in chat. These images are uploaded to **Cloudinary**, which stores and optimizes them for web display. The app then shares the secure image URLs with chat participants, enabling fast and reliable media sharing.
- **Password Recovery with Nodemailer:** A **forgot password** feature is implemented using **Nodemailer**. When users request a password reset, an email containing a secure link is sent to their inbox. This link allows them to safely reset their credentials and regain access to their account.
- **User Status Tracking:** Online users are tracked through Socket.IO. The server listens for user connections and disconnections, and updates all connected clients about the online status of participants. This adds a social dimension to the chat, showing who is currently active.

6.2 Dataset Integration

- **No Static Dataset Required:** Unlike applications that rely on labeled datasets for training models, ChatHive functions with **real-time user-generated data**. Messages, images, and user activity are dynamically handled and do not require preloaded datasets, making the system lightweight and scalable.
- **Real-Time Data Handling with Socket.IO:** ChatHive employs **Socket.IO** for real-time communication. When users send or receive messages, the data is immediately emitted and broadcasted to relevant sockets, allowing for **instantaneous message delivery** without storing the message in a static dataset.
- **Media Storage via Cloudinary:** Images shared through ChatHive are **uploaded to Cloudinary**, a cloud-based media storage platform. Instead of storing image data locally, the application saves the **Cloudinary image URLs** in the database. This approach reduces server load and improves performance while maintaining secure and optimized media delivery.

7. Conclusion

ChatHive is a real-time chat application built to provide users with a seamless and interactive communication experience. By utilizing **Socket.IO**, the app enables instant message delivery and live updates across connected clients. This ensures that users can chat without delays, creating a fluid and engaging environment similar to modern messaging platforms. The use of **real-time socket connections** is essential in building dynamic applications where responsiveness is a priority.

The application goes beyond just text messaging. Users can send and receive **images via Cloudinary**, a powerful cloud-based image storage and management service. This integration allows for efficient handling of media files, ensuring that images are compressed, optimized, and securely stored. It adds a visual dimension to conversations, enriching user interaction and opening the door for more expressive communication.

Security and user management have been given equal importance in Chat Hive. The platform implements **cookie-based authentication** to manage user sessions, ensuring that data remains secure and access is well-regulated. Moreover, the inclusion of a **password recovery system using Nodemailer** ensures that users can reset their credentials through email in case of login issues, enhancing the platform's overall reliability and user-friendliness.

Additionally, Chat Hive tracks and displays **online users in real-time**, allowing users to know who is currently active. This small but impactful feature helps foster more meaningful and timely conversations. Altogether, Chat Hive combines real-time technologies, cloud services, and user-focused functionality to deliver a robust communication tool. The project lays a solid groundwork for future upgrades such as group chats, message editing, typing indicators, or integration with voice and video APIs, positioning it well for broader and more immersive use cases.

8. Future Scope

1. **End-to-End Encryption & Enhanced Privacy:** Implement full end-to-end encryption (E2EE) for all messages, ensuring maximum security and user privacy. Add self-destructing messages, disappearing chats, and secure backup options to build user trust.
2. **AI-Powered Smart Replies & Autocomplete:** Integrate NLP-based AI models to suggest smart replies, message autocomplete, and sentiment-aware suggestions that improve user convenience and engagement.
3. **Emotion & Sentiment Detection in Messages:** Use real-time sentiment analysis to detect user emotion through text and emojis, allowing for mood-based reactions, UI adjustments, or support flagging in sensitive conversations.
4. **Audio & Video Calling Support:** Expand Chat Hive into a comprehensive communication tool by adding real-time audio and video calling with screen sharing and background blur capabilities.
5. **Voice Note Transcription & Translation:** Add voice message-to-text transcription and multilingual message translation to increase accessibility and usability across diverse audiences.
6. **Group Management & Threaded Conversations:** Enhance group chat functionality with roles (admin/moderator), threaded replies, polls, scheduled messages, and file pinning for better organization.
7. **Offline Messaging & Push Notifications:** Enable offline message composition and improve delivery with smarter push notifications, ensuring users are updated even during connectivity drops.
8. **Chatbot Integration for Automated Support:** Allow businesses to embed chatbots for automated customer support, appointment booking, and FAQs directly within the chat interface.

9. Bibliography / References

1. **GeeksforGeeks Articles:** In-depth articles on Node.js, Express, and MongoDB. These articles cover a wide range of topics, from basic concepts to advanced techniques, making them a valuable resource for developers.
2. **JWT Documentation:** Official documentation for implementing JWT authentication. This resource provides detailed information on how to generate, sign, and verify JWT tokens.
3. **Node.js Documentation:** Official resources for learning and implementing Node.js. The documentation covers everything from setting up a Node.js environment to building scalable backend systems.
4. **Socket.io Documentation:** Official guide for integrating WebSockets using Socket.io. This resource provides detailed information on how to set up and use Socket.io for real-time communication.

These references provide valuable insights and guidance for building the chat application, ensuring that the project is implemented using best practices and modern web technologies.